Srija Yadav Vuppula

+1 (706) 254-7688 | vuppulasrija16@gmail.com | LinkedIn | GitHub

GSoC-2026 : Metaflow Proposal

Metadata Service Request Improvements for Metaflow

About Me

The University of Georgia Athens, GA

Master of Science in Computer Science *Jan 2025 – Dec 2026*

- I am a Master's student in Computer Science at the University of Georgia. Most of my work has been around Python, backend development, Linux environments, automation, and debugging software systems. Before starting my Master's, I worked in an engineering role where Python and Bash were part of my daily work for automation, system support, and operational tasks. That experience made me comfortable with codebases that have multiple moving parts and with tracing problems across more than one layer.
- This Metaflow project stood out to me because it is practical. Instead of being about an isolated feature, it focuses on improving metadata request handling across the service API, provider layer, and client logic. That kind of work matches the way I like to learn: by understanding how a real codebase behaves end-to-end and then improving it in a way that is actually useful.
- To make sure I understood the project before writing this proposal, I worked through the code and setup first. Metaflow was installed locally, the tutorial flows ran successfully, and I joined the `#gsoc` Slack channel to start following the project more closely. I also cloned the `metaflow-service` repository, brought up the local development stack, and looked through the metadata endpoints and tests. Doing that early helped me write this proposal from actual exploration instead of only from the project description.
- Overall, I think I am a good fit for this project because it combines the kind of work I already enjoy: reading real code, tracing interactions between layers, and improving behavior in a way that stays simple, maintainable, and useful for users.

Project Description

- **Project**: Metadata Service Request Improvements for Metaflow
- **Problem Statement**: Metadata access is a core part of how users inspect runs, steps, tasks, and related resources in Metaflow. As the amount of stored metadata grows, collection endpoints that return large responses become more expensive in memory, bandwidth, and latency. On top of that, some filtering still happens on the client side after a larger response has already been fetched, which is less efficient than handling it directly in the request path.
- **Project Goal**: The goal of this project is to improve metadata request handling across the full path: the metadata service should support filtered and paginated responses more consistently, the client and provider code should handle paginated results correctly, backward compatibility should be preserved, and the final behavior should be backed by stronger tests.
- **Why This Matters**: This work matters because metadata access is something users depend on regularly. When that path becomes inefficient, the impact grows along with the number of runs and tasks in the system. Better request handling means smaller responses where possible, less unnecessary filtering in memory, and a cleaner interaction between the service API, provider interfaces, and client code.
- **Desired Outcome**: By the end of the project, metadata access should feel more predictable and efficient. Supported filters should be pushed into the request path whenever possible, paginated responses should be handled cleanly by the client, and the overall behavior should remain safe for existing users who still depend on current behavior.

What I Have Already Explored

- **Metaflow Local Setup**: Metaflow was installed locally and the introductory tutorial flows, including `00-helloworld` and `01-playlist`, were run successfully. That gave me a working environment and a better understanding of the local developer workflow before moving into the internals.
- **Main Repository Exploration**: The main Metaflow repository was then cloned so I could trace the metadata-related path through the client and provider layers. Rather than starting only from the project description, I wanted to see what already existed in the code and where the actual gaps were.
- **What I Found in the Main Codebase**: One of the first useful findings was that some metadata filtering behavior already exists in the client/provider path. That changed how I thought about the project. It became clear that this is not really about building everything from zero; the more useful direction is improving and standardizing what already exists so that filtering and bounded metadata access work more smoothly end-to-end.
- **Initial Community Interaction**: After the initial setup and reading, I introduced myself in the Metaflow GSoC Slack channel and shared that I was interested in the Metadata Service Request Improvements idea. The response there pointed me toward the `metaflow-service` repository, especially the GSoC-related issues and the local development stack, which helped narrow the focus to the actual service-side code relevant to this project.

- **Service Repository Setup**: The `metaflow-service` repository was cloned next, and the local development stack was brought up using the development Docker compose setup. A few environment and build issues had to be fixed along the way, which ended up being useful because it gave me a much better picture of how the repository is structured and how the services depend on each other.
- **Service-Side Exploration**: With the stack running locally, I verified the metadata endpoint and then dug into the metadata API routes, shared request utilities, and integration tests. That made it easier to see how pagination, filtering, grouping, and ordering are currently parsed and passed into the database layer.
- **Most Important Takeaway**: The biggest takeaway from all of this exploration is that the project makes the most sense as a request-path improvement project, not as a greenfield feature. Useful pieces already exist in different parts of the codebase, but they need to be made more consistent and better connected across the service API, provider layer, and client logic.

Technical Approach

- **Overall Strategy**: The project breaks naturally into four stages: audit the current metadata request path, improve or standardize the relevant service-side behavior, connect those changes through the provider and client code, and then validate everything with tests and documentation. That structure feels realistic for a medium-sized project and also keeps the work easy to review in smaller pieces.
- **Stage 1: Request Path Audit**: A clean map of the current request path is the right starting point. The first goal is to identify which collection endpoints are involved in metadata listing, which query parameters are already supported, which filters are already handled on the request side, and which filters are still being applied in memory on the client. That audit should make the actual implementation boundary much clearer and reduce the chance of solving the wrong problem.
- **Stage 2: Service-Side Improvements**: The next step is tightening the behavior of the metadata-service endpoints that matter for this project. The focus here is bounded and predictable request behavior: filtered and paginated access where needed, consistent handling across similar routes, and changes that stay close to the patterns already used in the service code.
- **Stage 3: Provider and Client Integration**: Once the service-side behavior is in place, the provider and client path needs to be updated so those improvements are actually usable. Paginated responses have to be consumed correctly, and supported filtering logic should move into the request layer where that is safe and useful. Backward compatibility stays important throughout this phase, since older clients still need to work correctly.
- **Stage 4: Testing and Validation**: Because this project affects more than one layer, testing is a major part of the implementation rather than something to add at the end. Integration coverage should be expanded around metadata listing behavior, with tests for pagination, filtering, ordering, and edge cases. The goal is not just to make the code pass once, but to leave behind behavior that is easier for maintainers to trust and extend later.

- **Implementation Philosophy**: The scope should stay practical. A smaller, correct, well-tested improvement is much more valuable than a larger redesign that becomes difficult to finish or maintain. For a 175-hour project, that tradeoff matters.

Detailed Implementation Plan

- **Phase 1: Baseline Audit**: The metadata request path across both the main Metaflow repository and the service repository needs to be reviewed carefully. This phase focuses on identifying the endpoints, helper functions, and provider/client flows involved in metadata listing and metadata-based filtering. Alongside that, I want to keep a short internal note describing the current behavior, the gaps I found, and which parts should be targeted first.

- **Phase 2: Service Endpoint Work**: Filtered and paginated behavior should then be implemented or standardized on the most relevant metadata collection endpoints. The point of this phase is to make the service response path more bounded and more consistent, while still staying aligned with the request parsing and response patterns already used in the repository.
- **Phase 3: Provider and Client Updates**: The provider and client path then needs to be updated so paginated responses can be requested and consumed correctly. Wherever filtering is currently done in memory and a safe request-level alternative already fits the design, that logic should be moved into the request path. Backward compatibility checks are an important part of this phase as well.
- **Phase 4: Integration Testing**: Integration coverage should expand to include pagination, filtering, ordering, and the ways these features interact with each other. Rather than only relying on isolated artificial cases, the tests should reflect realistic behavior and cover the main request paths touched by the implementation.
- **Phase 5: Cleanup and Documentation**: The final phase is reserved for cleanup: improving readability, removing temporary debugging work, and adding useful comments or notes where they help. If time allows, I also want to document the expected metadata request behavior more clearly so future contributors can understand the design decisions more quickly.

Expected Deliverables

- **Core Deliverable 1**: Improved and more consistent filtered and paginated request handling on the targeted metadata-service collection endpoints.
- **Core Deliverable 2**: Provider/client support for consuming paginated metadata responses correctly.
- **Core Deliverable 3**: Reduced reliance on in-memory filtering for the targeted metadata access path, with request-level filtering used where appropriate and safe.
- **Core Deliverable 4**: Expanded integration and regression test coverage for metadata listing behavior, including pagination, filtering, ordering, and compatibility-related cases.
- **Core Deliverable 5**: Implementation notes and documentation improvements that explain the supported behavior and design choices clearly enough for maintainers and future contributors.

Potential Challenges and Mitigation

- **Backward Compatibility**: The biggest technical risk here is changing metadata behavior in a way that breaks older Metaflow clients. Compatibility needs to be treated as a requirement from the beginning, with careful validation and safe fallback logic where needed, instead of being left as a final check.
- **Cross-Layer Complexity**: Since the project spans the service API, provider layer, and client logic, a change that looks correct in one place can still break the full request path. Smaller end-to-end milestones should help reduce that risk better than large isolated changes.
- **Scope Control**: Metadata-related work can easily turn into a larger API redesign discussion. Keeping the work centered on request-level filtering, paginated metadata handling, stronger tests, and backward compatibility should help keep the project within a realistic medium-project scope.
- **Environment and Service Setup Overhead**: Service-side work can slow down because of local environment issues, dependencies, or Docker-related problems. Since I have already spent time getting the local stack running, that risk is lower now, and smaller milestones should make it easier to keep moving even when setup issues appear.

- **If I Fall Behind**: If scope needs to be reduced, the priority order is clear: service-side bounded response behavior first, provider/client handling next, request-level filtering on the highest-impact path after that, and extra cleanup or stretch improvements last. That way, the final result still remains a correct and useful core improvement.

Weekly Timeline

- **Community Bonding Period**: This period is mainly for reconnecting with mentors, confirming the final scope, reviewing the important code paths again, and breaking the project into smaller development tasks. Keeping the local setup stable during this time should make it easier to begin implementation immediately once the coding period starts.
- **Week 1**: Week 1 is dedicated to a structured audit of the metadata request path. The main outputs here are a clear list of the key endpoints, utilities, and provider/client flows involved in the project, along with a summary of the current behavior and the gaps between what exists now and what the project needs.
- **Week 2**: The focus in Week 2 is narrowing the implementation boundary with mentor feedback. That includes deciding which metadata listing paths belong in the core scope for the summer and which ones should stay out of scope. Baseline tests for current behavior can also be reviewed or added here if needed.
- **Week 3**: Service-side implementation begins in Week 3 with the first targeted metadata collection endpoint. Work in this phase centers on adding or standardizing filtered and paginated behavior there, along with the first related integration tests.
- **Week 4**: Week 4 continues the service-side work across the remaining metadata listing paths that are part of the project scope. At the same time, the response behavior should be checked for clarity, boundedness, and consistency, and test coverage should expand for both common and edge cases.
- **Week 5**: Provider and client integration becomes the priority in Week 5. The main goal is to update the relevant path so paginated responses can be consumed correctly and to validate the first full end-to-end request flow from the service layer through to the client.
- **Week 6 – Midterm Milestone**: By midterm, at least one major metadata listing path should work end-to-end with bounded response handling and supporting tests. This milestone is meant to be something concrete and demonstrable, not just a collection of partial internal changes.
- **Week 7**: Request-level filtering work takes focus in Week 7. Wherever there is a safe and supported alternative to in-memory filtering, that behavior should move into the request path, with more focused regression tests added around the updated flow.
- **Week 8**: Integration coverage expands further in Week 8 to cover pagination, filtering, ordering, and the ways these features interact. Edge cases such as empty results, invalid request combinations, and compatibility-sensitive scenarios also need attention here.
- **Week 9**: Week 9 is mainly about strengthening backward compatibility handling and cleaning up implementation details. Code readability, tighter tests, and feedback from earlier review rounds should all be addressed during this phase.
- **Week 10**: Documentation and cleanup take priority in Week 10, preparing the implementation for broader review and making sure the test suite clearly reflects the intended metadata request behavior.
- **Week 11 – Code Freeze**: No new feature work should be added during code freeze. The time here is reserved for bug fixes, review feedback, test stability, and documentation polish so the implementation can settle before final submission.
- **Week 12**: Final validation, final cleanup, and submission wrap up the last week, along with a concise summary of the work completed.

Other Commitments

- **Availability**: During the coding period, I expect to spend around 15–18 hours per week consistently on this project. That estimate feels realistic and sustainable, and it can stretch a little during lighter academic weeks if needed.

- **Communication**: Regular progress updates, focused technical questions, and early discussion of partial findings are all things I am comfortable with. Since this project spans multiple layers, checking assumptions early matters a lot more than trying to solve everything alone first.
- **Schedule**: At the time of writing, I do not expect any major long-duration conflicts that would prevent steady weekly progress. If academic work overlaps with a milestone, I would rather communicate it early and adjust the plan than let it silently affect the project.
- **Contribution Beyond GSoC**: This project would also be a strong starting point for longer-term contributions, since it gives direct experience with Metaflow's service API, provider interfaces, and client logic. Ideally, the summer work would turn into a longer contribution rather than ending with GSoC itself.

Final Note

- The goal of this project is not only to add one feature, but to make the metadata request path in Metaflow more efficient, more consistent, and better tested. Time has already gone into setting up the local environment, running Metaflow tutorials, tracing the relevant code paths, exploring the service repository, and understanding how the current request flow works.
- That early exploration made the proposal much more concrete. Parts of this behavior already exist, and the value now is in improving them in a way that stays consistent, backward-compatible, and maintainable.
- I would be excited to contribute this work to Metaflow during GSoC 2026 and continue contributing beyond the program.